

Relatório lista 3 estruturas de dados

De todos os algoritmos de ordenação estudados, dois foram implementados neste trabalho: o insertion sort e o merge sort. Para ambos, foram utilizadas duas funções auxiliares com o intuito de comparar automaticamente elementos da classe jogadores, a `comesBefore` e a `comesBeforeOrEqual`. Ambas seguem os critérios estabelecidos no enunciado, ou seja, comparam o score e em caso de igualdade usam o timestamp como desempate. Desse modo, o insertion sort foi feito de modo habitual, com `comesBefore` para comparar e ir empurrando para trás os jogadores desordenados.

Já o merge sort envolveu mais elaboração, pois para conservar a assinatura da função `sortByScoreMerge` que o professor passou, esta função chama internamente o método interno `mergeSort`, que efetivamente realiza o merge sort recursivamente, conforme visto em aula, em conjunto com o outro método interno `merge`. Essa decisão introduz um custo adicional de cópia, mas não altera a ordem assintótica do algoritmo, que permanece $O(n \log n)$.

Por fim, a ordenação dos jogadores foi aproveitada diretamente na função de formação de grupos, pois agrupa jogadores de níveis semelhantes mais próximos uns dos outros dentro do array, permitindo que a função `formGroup` procure grupos válidos analisando apenas janelas consecutivas de tamanho `groupSize`. Em cada janela, basta verificar se a diferença entre o score do último e do primeiro jogador é menor ou igual ao valor `delta`. Assim, a ordenação poupa o trabalho de comparar todos os jogadores contra todos ou testar todas as combinações possíveis.

Quanto às análises de custos das funções principais, a `insert`, que não possui laços ou recursões, tem custo constante e, portanto, é $O(1)$. A função `removePlayer` percorre o array inteiro procurando o id informado e, caso encontre, desloca para a esquerda todos os elementos em uma unidade ao remover. Logo, possui complexidade $O(n)$. A `sortByScoreInsertion`, por se tratar de um insertion sort, envolve varrer o array com um loop dentro de outro, o que gera em geral $O(n^2)$. Já o `sortByScoreMerge`, naturalmente por envolver quebrar o array em duas partes $\log n$ vezes e depois ir reagrupando todos os elementos é naturalmente $O(n \log n)$. Um ponto a ser destacado nesta função é que como ela foi implementada chamando internamente outra e realizando uma cópia do array ordenado inteiro no final, ela tem um custo adicional proporcional ao tamanho do array $O(n)$, porém assintoticamente esse custo é majorado pelo $O(n \log n)$. Quanto a função `formGroup`, ela envolve percorrer os possíveis grupos de jogadores com um laço e no pior dos casos pode analisar a fila toda. Depois de encontrar um grupo válido, copia todos os jogadores para um novo array. Como o tamanho dessa cópia costuma ser bem menor que o da fila inteira, em geral o custo é $O(n)$. Por fim, a função `getWaitingPlayers` envolve verificar se a fila está vazia e copiar todos os jogadores para um novo array,

resultando em geral em $O(n)$. Outras funções que efetuam comparações como `comesBefore`, por serem verificações simples, em geral são $O(1)$.

n (jogadores)	Insertion Sort (ms)	Merge Sort (ms)	Razão (IS / MS)
1	19,849	0,6431	~31x
5	302,87	28,879	~105x
10	1180,06	70,351	~168x
50	30161,2	334,996	~900x
100	125216	669,547	~1.870x

Os testes de desempenho foram realizados no `main.cpp`, utilizando dois objetos `MatchMaking` com os mesmos dados para comparar o insertion sort e o merge sort em condições idênticas. Os jogadores foram gerados com scores em ordem decrescente, representando o pior caso para o insertion sort. O tempo foi medido utilizando `std::chrono::high_resolution_clock`, considerando apenas a execução da ordenação. Os testes foram feitos com entradas de 1000, 5000, 10000, 50000 e 100000 jogadores.

Os resultados mostraram uma diferença de desempenho crescente entre os algoritmos. Enquanto o insertion sort levou mais de 3 minutos para ordenar 100000 jogadores, o merge sort realizou a mesma tarefa em menos de 0,67s. A diferença entre os tempos aumenta significativamente conforme o tamanho da entrada cresce, tornando o insertion sort inviável para grandes volumes de jogadores.

Os resultados observados são coerentes com as complexidades teóricas dos algoritmos. Conforme visto antes, o insertion sort possui complexidade $O(n^2)$ e apresentou crescimento quadrático no tempo de execução, especialmente no pior caso utilizado nos testes. Já o merge sort possui complexidade $O(n \log n)$ e manteve crescimento muito mais eficiente conforme o tamanho da entrada aumentou. Apesar do merge sort exigir memória adicional $O(n)$ para os arrays auxiliares, o ganho de desempenho justifica sua utilização em sistemas de matchmaking com muitos jogadores.